

ASSIGNMENT-1

Java Introduction

1. Java is an object oriented Programming language & High level language which is developed by James Gosling and his team Sun Microsystems in 1995. It is known for platform Independent, Security and Simplicity.
2. Features of Java:
 1. Platform Independent:

Java is platform Independent as it contains JVM which can be runned on any System regardless of the OS.
 2. Object Oriented:

It Supports Oop Concepts where the classes, object. Inheritance, abstraction, encapsulation these support the Code reusability and organization
 3. Multithreading:

It allow Simultaneously working on multiple task in a given period of time.
Concurrent occurrence of multi tasks
 4. Robutness:

As it has garbage Collector which helps in memory managment, Expection handling leads to high performance.
 5. Simplicity:

Easy to learn and use, making it acessible to all the programmers.
3. Java is platform Independent as it can be Compiled into bytecode on any System Contain-ing JVM. It has Write Once, Run Anywhere Capability.

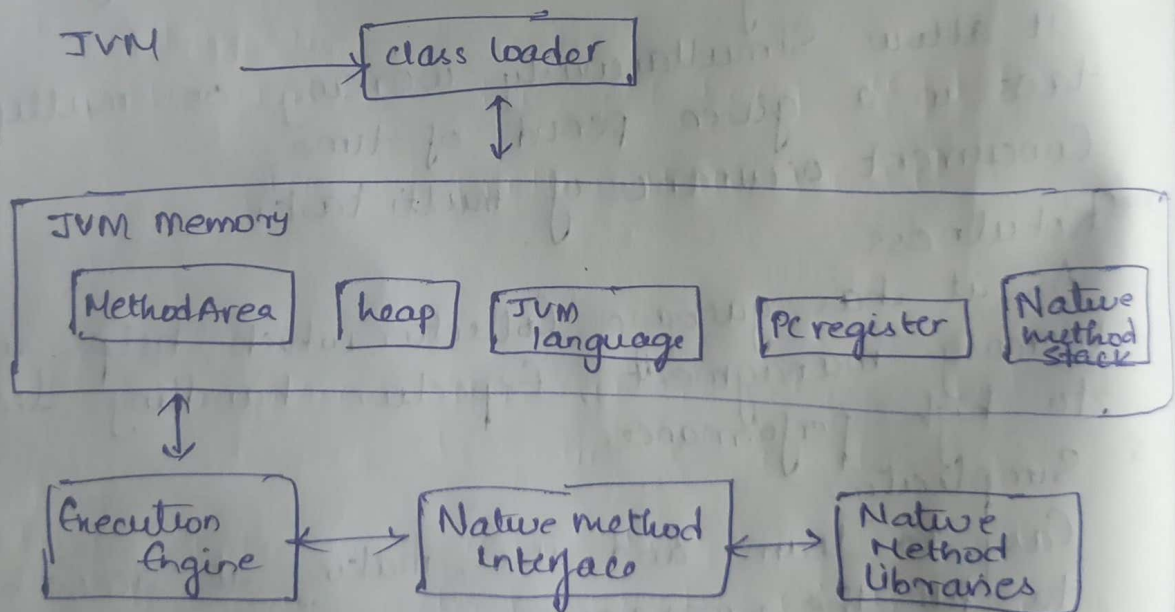
JVM/JRE/JDK Architecture

4. → JVM - Java Virtual Machine.
It is used for the code to be compiled into the bytecode which make the Java to be platform Independent.
It helps to run the Java byteCode

→ JRE - Java RunTime Environment.
It is used to run Java program on the System. It also includes the libraries and Jvm. Platform Dependent

→ JDK - Java Development Kit.
It is develop the Java application. It is combination of both JVM and JRE. It is platform Dependent

5. JVM Architecture:



① class loader:

It is responsible for loading class file into the memory area.

It contain 3 stages

- i. loading
- ii. linking
- iii. initialization

- i. loading: It loads the class Name, Variable information whenever Jvm is loaded.
- ii. linking: It helps in attaching the loaded class files into the memory area.
- iii. Initialization: All static Variables are assigned with their values defined in code and static block.

② Jvm Memory Area:

Method Area:

It is used to store class Name, Variable information, methods. It is shared memory resource.

All static block / variable / references

Heap Area:

Information of all objects is stored in heap.

It is shared resource. It is for instance variable.

Stack Area:

All local variables of method are stored and every thread, with its run-time stack is stored here. It is not sharable.

PC Register:

Store Address of current execution instruction of a thread. Each thread has different PC.

Native method Stack:

Each thread creates a native stack. Its information is stored here.

③ Executive Engine:

It executes the class (bytecode). It reads the byte code line by line, uses data and information present in various memory areas & executes instructions.

JIT: It is used to increase efficiency as it compiles entire bytecode and to native code whenever repeated code is being used again to save and no need to interpret it multiple times.

Interpreter:
It reads the code line-by-line which makes more time and re-run of the multiple-time method reads.

Garbage-Collector:
It is used to clean up the unused object references such that it frees up the memory from the Memory Area.

6. JIT Compiler:

- It helps in compilation of frequently used ^{byte} code to machine code at the runtime.
- It speeds up helps in increase of performance and optimization.

JAVA PROGRAM STRUCTURE

7. Hello World! Program

```
class Program {  
    public static void main (String args[])  
    {  
        System.out.println('Hello World!');  
    }  
}
```

8. • Class Program → Here it is class declaration
Program is a class name

- Public static void main (String args[]) → It is the main method which is the entry point of the program.

To execute a program JVM Search for this main

- System.out.println → It is used to print output

where System is a class
out is a object
println is a method.

9. Main method: It is the entry point. At JVM Search for the main which is to start the execution of the program.

Keywords & Identifiers

10. Keywords:

Keywords are the reserved name that are used to perform certain action or specified task.

Ex: int $\rightarrow$ This is used for only the numbers.

char $\rightarrow$ This is used for characters, Digits [0-9]
Alphabets [a-z] [A-Z]

11. 10 Java Keywords:

int: Only allow number

char: Allow special chara, Digits [0-9], Alphabet [a-z] [A-Z]

for: To perform loop

while: To perform loop when doesn't only how many time

Do: Used to get executed atleast once

Boolean: 2 Values True / False

Short: Allow number, with less space

long: Allow number, for large digits

Double: Used for the precision of decimal before 6

Break: To break the on-going program, stop execution.

12. Identifiers:

They are names that are used to identify the values at the time of assigning.

Rules: Cannot start with Capital letter

No keyword can be used

Support only special char like \$, -

Always can be started with small letters

ex: age, -age, \$age, agePerson

Variable & Data Type

13. Primitive & Reference Data type:

Primitive Data Types:

They are the datatype that are inbuilt in Java and cannot create a object.

They are used for declaration & Assigning the type of Data ex: int, short, boolean etc

Reference / Non-Primitive.

These datatypes are used for the creation of object which can be instantiated.

They always start with Capital letters.
ex: String, ~~Object~~, Array.

14. Variables of All primitive Data types:

A variable is declared as

DataType VariableName
or

DataType VariableName = Value (Assigning)

int age = 20; - 4 byte

Double price = 28.6d; - 8 byte

boolean check = True; - 1 bit

~~Short name = 'Tin'~~

Short value = 10 - 2 bytes

Float discount = 10.8f; - 4 bytes

long Rank = 2581496008f; - 8 byte

char value = 'A'; - 2 bytes

byte b = 10; - 1 byte

15.

```
import java.util. Scanner;  
class Student {
```

```
    public static void main (String args[]) {
```

```
        Scanner sc = new Scanner (System.in);
```

```
        System.out.println ("Enter name");
```

```
        String name = sc.nextLine();
```

```

System.out.println('Enter age');
int age = sc.nextInt();
System.out.println('Enter grade');
char grade = sc.nextChar(), CharAt();
System.out.println('Name is' + name);
System.out.println('Age is' + age);
System.out.println('Grade is' + grade);
}
}

```

Type Casting:

16. Implicit & Explicit Casting:

Implicit / Widening Casting:

They are used to convert small datatypes into large Datatype. It is done Automatic by JVM.

byte → short → char → int → long → float → Double

Explicit Casting / Narrowing

They are used to Convert large Datatype into Smaller Datatype. It is done manually. JVM Can't do it.

Double → float → long → int → char → short → byte

Ex: Implicit → int value = 10;
float Value1 = value;

Explicit → int value = 10;
short Value1 = short (value);

17. Both type of casting:

public class Casting {

public static void main (String args[]) {

```

Implicit {
    int a = 10;
    Double b = a;
    System.out.println(b); (Double
                             + type)
Explicit {
    Double c = 99.9;
    int d = (int) c;
    System.out.println(c); → 99
}
}

```

18. Narrowing Conversion fails:
If this occur there may be chance of data loss, raise of error and unexpected behaviour.
As big data tries to fit in small size this undergo overflow of data.

Operators

19. $==, =$

$==, =$
 $== \rightarrow$ it is used to compare the reference types
 of 2 values

```
int a=5;
int b=5; → both occupy same reference
if (a==b)    So it gives True
```

= $\rightarrow$ It is used to assign a value to a variable

int a = 5; Assign 5 to a variable;

20. Program:

```
import java.util.Scanner;  
class operator {
```

```
public static void main (String args[]) {
```

```
Scanner sc = new Scanner(System.in);
```

```
System.out.print ("Enter value a");  
int a = sc.nextInt();
```

```
int a = sc.nextInt();
```

```
System.out.println("Enter Value b");
int b = scanner.nextInt();
```

```
int b = sc.nextInt();
```

if $(a > b) \{$

```
System.out.println 'Added' + (a+b);
```

3

```
else if (a == b) {
```

'Multiplied' +

```
System.out.println(a*b);
```

உலகம்

1. Subtract +

```
System.out.println(a-b);
```

3 3 3

q/p:	Enter value a	5	4
	5		
	Enter value b	5	5
	4		
	Added a	multiplied 25	Subtracted -1.

21. logical operator : they are used to manipulate boolean types

logical operator are &&, ||, !

&& → If both values are True then True
any 1 value fails then false

|| → If any 1 value is True return True

! → Return the opposite of the boolean Type

boolean a = 'True' → if (!a) → return false

Control Flow

22. largest of 3 numbers: using if-else

```
import java.util.Scanner;
```

```
class largest {
```

```
    public static void main (String args[]) {
```

```
        Scanner sc = new Scanner (System.in);
```

```
        int a = sc.nextInt();
```

```
        int b = sc.nextInt();
```

```
        int c = sc.nextInt();
```

```
        if (a > b && a > c)
```

```
        {
            System.out.println (a + ' is largest');
        }
```

```
        else-if ( b > c )
```

```
        {
            System.out.println (b + ' is largest');
        }
```

```
        else
        {
```

```
            System.out.println (c + ' is largest');
        }
```

```
    }
```

```
}
```

o/p:

5
3
6

6 is largest

23. Switch Case is also executes as if case but when there are many Condition we prefer switch case then if Condition statement

Syntax: Switch (Case):

```
{  
    Case1: {  
        }  
    Case2: {  
        }  
    Case3: {  
        }  
}
```

24. loop to print 10-1

```
class loops {
```

```
    public static void main (String args[]) {
```

```
        for (int i=10; i>0; i--)  
        {
```

```
            System.out.println(i);
```

```
        }
```

```
    }
```

Arrays

25. Array is the Collection of homogenous data types. They are used to store similar type of data in stack.

It has fixed-size.

They are stored in contiguous memory location

26. Program to find Sum of element in an array.

```
import java.util.Scanner
Public Class Sum {
    Public Static void main (String args[]);
    Scanner sc = new Scanner (System.in);
    System.out.println ('Enter Array Size');
    int size = sc.nextInt();
    System.out.println ('Enter Array Values');
    int arr[] = new int[];
    for (int i = 0; i < n; i++)
    {
        int arr[i] = sc.nextInt();
    }
    for (int i = 0; i < n; i++)
    {
        int sum = 0;
        sum = sum + arr[i];
    }
    System.out.println ('Sum of Array:', sum);
}
```

o/p: Enter Array Size
3
Enter Array Value
1
5
6
Sum of Array: 12

27. A Two Dimensional Array

int arr[][] = new int[][] ← Declare

or

int arr[][] = { {1, 2}, {3, 4} }

System.out.println (arr[1][0]);

o/p → 3